idea-harness

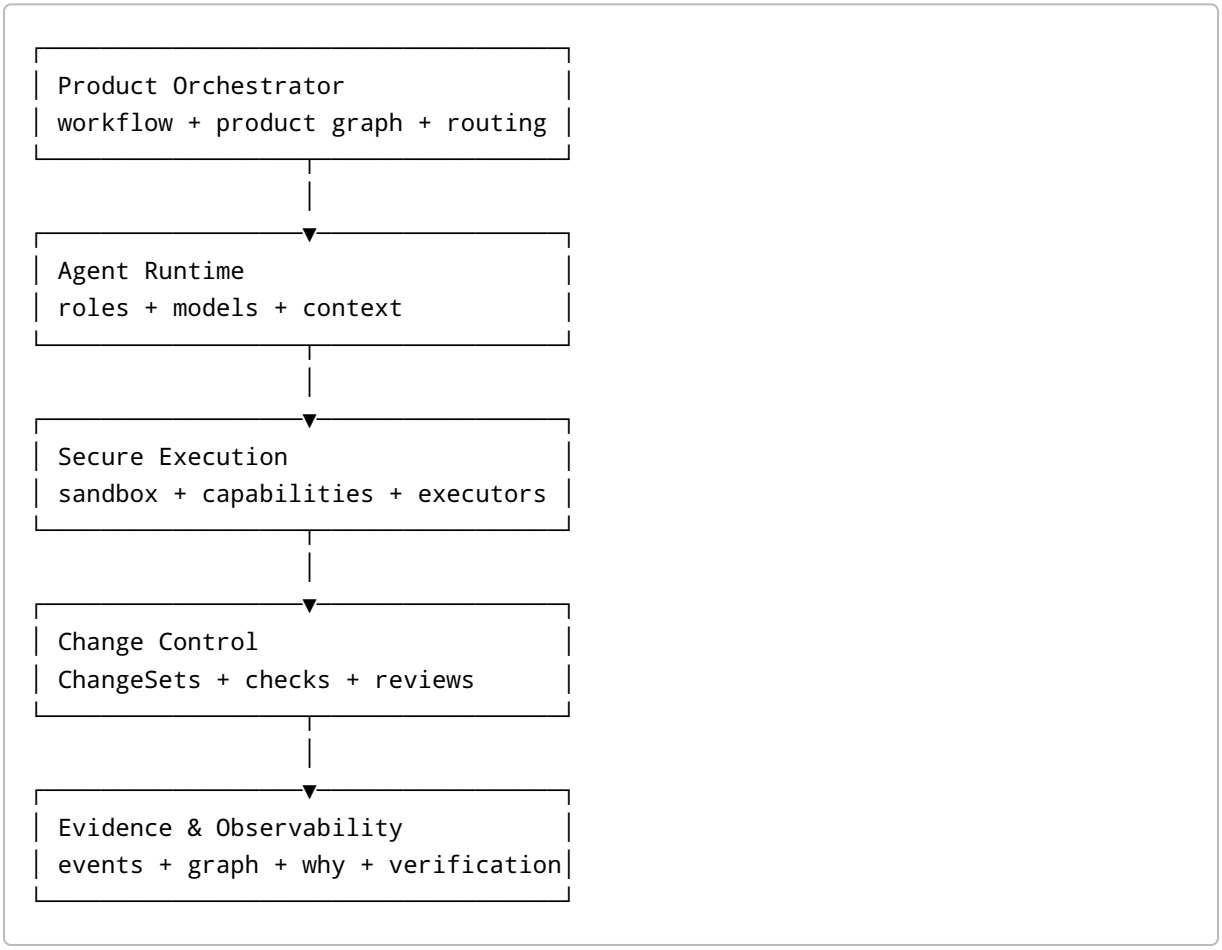
Technical Design Document

Version: 1.2
Status: Implementation Baseline
Target: v0.1
Related: PRD v1.1

---

1. Architecture Summary

idea-harness is implemented as five major systems:



The implementation must remain simple enough to run locally while preserving strict boundaries between:

```
reasoning
authorization
execution
workspace mutation
evidence
```

---

2. Technology Baseline

Recommended v0.1 stack:

```
Daemon / CLI       Go
Web UI             React + TypeScript
Database           SQLite
Large content      Content-addressed BlobStore
Live UI updates    SSE
Configuration      YAML + JSON Schema
API                REST
API schema          OpenAPI
Sandbox             Docker initially
External executor  OMP initially
Observability       OTel-compatible internal model
```

---

3. Daemon Model

Use one daemon per local user.

```
idea daemon
  ├── Project A
  ├── Project B
  └── Project C
```

Benefits:

- one API
- one UI
- one provider manager
- one executor supervisor
- one process lifecycle
- future multi-project dashboard

Default network binding:

```
127.0.0.1
```

Remote binding requires explicit configuration.

---

4. Global and Project Storage

```
~/.local/share/idea-harness/

global.db

projects/
  <project-id>/
    state.db
    blobs/
    snapshots/
    changesets/
    workspaces/
    cache/
    logs/
```

`global.db` stores:

- project registrations
- project paths
- daemon metadata
- global config
- provider metadata

Project DBs are fully isolated.

---

5. Portable Project Configuration

```
project/
├── .idea-harness/
│   ├── project.yaml
│   ├── harness.yaml
│   ├── skills.lock
│   ├── agents/
```

```
│   ├── standards/
│   ├── workflows/
│   ├── policies/
│   ├── skills/
│   ├── schemas/
│   └── artifacts/
│
└── source files...
```

Runtime state must not pollute the repository.

---

6. Core Architectural Invariants

1. Git is optional.
2. Workers cannot directly mutate the canonical workspace.
3. Every source mutation is a ChangeSet.
4. Every ChangeSet receives independent review.
5. Every material plan receives independent review.
6. Authors cannot self-review.
7. The deterministic engine controls authorization.
8. Models cannot grant capabilities.
9. Skills are non-executable.
10. Tools and Executors explicitly declare capabilities.
11. Security is enforced outside prompts.
12. Large content does not live inline in SQLite.
13. Relational state is authoritative.
14. Events accompany every meaningful durable state transition.
15. Events and relational state update atomically.
16. Every Run produces immutable provenance.
17. Project ID scopes every security-sensitive operation.
18. User edits are never silently overwritten.
19. Canonical workspace writes are serialized.
20. Infrastructure and semantic failures have distinct recovery.
21. Reviews may vary in depth but cannot disappear.
22. Draft agent output is lower-trust than approved output.
23. Human-required approvals cannot be satisfied by agents.
24. Security failures fail closed.

---

7. v0.1 Domain Model

Core domain:

```
Project
Initiative
Workflow
StageRun
Task

Assertion
Requirement
AcceptanceCriterion
Decision
Artifact

ChangeSet

Run
ExecutionAttempt
Review
Finding
VerificationEvidence

GraphEdge
Approval
Operation
Event
```

Supporting technical objects:

```
RiskProfile
ContextBundle
RunManifest
ProjectProfile
ApplyJournal
```

Avoid creating independent aggregates until behavior requires them.

---

8. Product Graph

Graph nodes:

```
Source
Assertion
Requirement
```

```
AcceptanceCriterion
Decision
Artifact
Task
ChangeSet
Test
Review
Verification
```

Graph edges:

```
supports
contradicts
derived_from
depends_on
implements
addresses
tests
verifies
reviews
supersedes
invalidates
produced_by
consumes
```

Storage:

```sql
CREATE TABLE graph_edges (
    id TEXT PRIMARY KEY,
    project_id TEXT NOT NULL,
    from_type TEXT NOT NULL,
    from_id TEXT NOT NULL,
    edge_type TEXT NOT NULL,
    to_type TEXT NOT NULL,
    to_id TEXT NOT NULL,
    metadata_json TEXT,
    created_at TEXT NOT NULL
);
```

Indexes must always include project scope.

---

9. Current Understanding Projection

`Current Understanding` is not stored as an independent source of truth.

It is derived from approved:

```
Requirements
Assertions
Decisions
Artifacts
Open Questions
Risks
```

Implementation:

```go
type CurrentUnderstandingService interface {
    Build(
        ctx context.Context,
        initiativeID UUID,
    ) (CurrentUnderstanding, error)
}
```

It may use an LLM for concise synthesis, but the structured facts being summarized are deterministic inputs.

Every rendered statement must retain source references.

---

10. Recommended Next Move

Produced by:

```
Workflow state
+
unresolved decisions
+
blocking gates
+
risk
+
dependency graph
```

The deterministic engine first generates candidate actions.

The Orchestrator Agent may rank or explain them.

The agent cannot create an action that violates workflow policy.

---

11. What Changed

Changes are derived from event and graph history between user checkpoints.

Checkpoint inputs:

```
last_seen_event_sequence
OR
explicit timestamp
```

Semantic summarization groups events into:

```
Product
Design
Build
Quality
Security
```

Raw events remain available.

---

12. Workflow

Human macro-phases:

```
Explore
Define
Design
Build
Verify
```

Underlying DAG:

```
Framing
   ↓
Research
   ↓
Product Definition
   │
   ├── Architecture
   ├── Product Design
   ├── Verification Design
   └── Threat Model?
          ↓
       Convergence
          ↓
    Execution Plan
          ↓
    Implementation
          ↓
     Verification
```

Stage conditions remain declarative.

---

13. StageRun State Machine

States:

```
queued
ready
running
review
blocked
done
skipped
cancelled
```

Example legal transitions:

From	To	Preconditions
queued	ready	dependencies satisfied
ready	running	scheduler capacity
running	review	required output exists

From	To	Preconditions
running	blocked	semantic blocker
review	done	required gate passed
review	running	revision required
blocked	ready	blocker resolved
non-terminal	cancelled	authorized

State transitions must occur through domain methods, never arbitrary persistence updates.

---

14. Operation

Long-running application-level command.

```
type Operation struct {
    ID          UUID
    ProjectID   UUID
    Type        string
    SubjectType string
    SubjectID   UUID
    Status      OperationStatus
    Error       *HarnessError
    StartedAt   time.Time
    CompletedAt *time.Time
}
```

Used for:

- API commands
- CLI commands
- long-running workflow actions

---

15. Run

Generic execution record.

```
RunKind:
model
executor
```

```
review
check
tool
```

```
type Run struct {
    ID           UUID
    ProjectID    UUID
    InitiativeID *UUID
    StageRunID   *UUID
    TaskID       *UUID
    Kind         RunKind
    Status       RunStatus
    ManifestID   UUID
}
```

---

16. ExecutionAttempt

Executor-specific durable execution.

Needed so daemon recovery can distinguish:

```
executor already completed
```

from:

```
executor must be rerun
```

An OMP execution result remains recoverable even if the daemon crashes before ChangeSet creation.

---

17. BlobStore

Large content uses a content-addressed store.

```
type BlobStore interface {
    Put(ctx context.Context, data []byte) (Digest, error)
    Get(ctx context.Context, digest Digest) ([]byte, error)
```

```
    Has(ctx context.Context, digest Digest) (bool, error)
}
```

Used for:

- artifacts
- captured research
- model outputs
- context
- review bodies
- patches
- changed files
- large logs

Local layout:

```
projects/<id>/blobs/sha256/<digest>
```

---

18. SQLite

Project state stores metadata, relationships, lifecycle, and indexes.

Do not store large body content inline.

Recommended:

```
journal_mode=WAL
foreign_keys=ON
busy_timeout=5000
synchronous=NORMAL
```

Daemon is the sole normal writer.

---

19. State + Event Transaction

idea-harness is not full event sourcing.

Authoritative state lives in relational tables.

Mutation:

```
BEGIN IMMEDIATE

validate current state
apply state mutation
increment event sequence
append event

COMMIT
```

Events provide:

- audit
- causality
- notifications
- debugging
- eventual replay

---

20. Event

```
type Event struct {
    ID          UUID
    ProjectID   UUID
    Sequence    int64
    Type        string
    Aggregate   string
    AggregateID UUID
    Actor       ActorReference
    Correlation *UUID
    Causation   *UUID
    OperationID *UUID
    Payload     json.RawMessage
}
```

Large payloads reference BlobStore digests.

---

21. Assertions

```
type Assertion struct {
    ID           UUID
    HumanID      string
```

```
    InitiativeID UUID
    Kind         AssertionKind
    Statement    string
    Confidence   Confidence
    Status       AssertionStatus
    SourceRefs   []ObjectReference
}
```

Kinds:

```
fact
assumption
hypothesis
```

Statuses:

```
candidate
accepted
disputed
retracted
superseded
```

Human corrections trigger impact analysis.

---

22. Requirements

Stable logical Requirement identity with version history.

Material intent changes create new requirements rather than silently rewriting old identity.

Requirement deletion/removal must preserve historical lineage.

---

23. Decisions

Approved Decisions become immutable.

Reversal:

```
DEC-8
superseded_by
DEC-14
```

This preserves causal history.

---

24. Artifacts

Metadata in SQLite.

Content in BlobStore.

Materialized files optionally written into:

```
.idea-harness/artifacts/
```

Artifact versions are immutable.

Status and freshness are separate.

```
status:
draft
valid
invalid
superseded

freshness:
current
potentially_stale
stale
```

---

25. Semantic Artifact Diff

Artifact-specific comparators should compare structured components before falling back to source diff.

Examples:

- Requirement additions/removals
- changed priority

- Decision supersession
- new risk
- modified acceptance criterion

Raw markdown diff is secondary.

---

26. Impact Analysis

When an upstream object changes:

```
Graph traversal
   ↓
Potential dependents
   ↓
Freshness = potentially_stale
   ↓
Impact classification
```

Classification:

```
unaffected
revalidate
revise
obsolete
```

The semantic agent may classify ambiguous cases.

The deterministic engine applies resulting state.

---

27. Role Contracts

Declarative roles define:

- competencies
- required behaviors
- required outputs
- rubrics
- default tools
- required capabilities
- eval suite

Prompt generation consumes the Role Contract.

---

28. Expert Routing

v0.1 deterministic signals:

```
has UI              → Product Design
auth/authz           → Security
schema migration     → Database specialist
public API/SDK       → DevEx
deployable service   → SRE
payments             → Payments
AI/agent feature     → AI Security
```

Agent-proposed consultations require orchestrator authorization.

---

29. Quality Design

Verification Design runs during Design rather than after it.

Output maps:

```
AcceptanceCriterion → Verification strategy
```

Implementation tests must be explicitly associated with acceptance criteria before being considered verification evidence.

---

30. Threat Modeling

High-risk Product Definitions may trigger a ThreatModel StageRun.

Security-derived requirements enter the ordinary Requirement model.

Threat modeling must occur before design convergence when triggered.

---

31. Risk

v0.1 RiskProfile:

```
type RiskProfile struct {
    Tier    RiskTier
    Signals []RiskSignal
    Reason  []string
}
```

Tiers:

```
low
medium
high
critical
```

Avoid numerical scores.

Deterministic security signals establish a minimum tier.

Semantic analysis may increase the tier.

---

32. Reviews

Review is long-lived across cycles.

```
REV-12
  Cycle 1
    subject PLAN v1
    failed
  Cycle 2
    subject PLAN v2
    passed
```

Review subjects:

```
artifact
execution plan
ChangeSet
```

```
task outcome
verification
```

---

33. Review Independence

Engine enforces:

```
reviewer run != author run
reviewer session != author session
```

Different model family is preferred, not required.

The author cannot directly resolve the Review as approved.

---

34. Finding

```
type Finding struct {
    ID             UUID
    ReviewID       UUID
    Severity       Severity
    Confidence     Confidence
    Category       string
    Finding        string
    Evidence       []EvidenceReference
    Impact         string
    Recommendation string
    Status         FindingStatus
}
```

Tracking rejected findings is required to measure reviewer precision.

---

35. Review Budget

Risk determines:

- maximum model calls
- context scope
- specialist reviewers

- human requirement

Example:

```
review:
  low:
    maxModelCalls: 1

  medium:
    maxModelCalls: 2

  high:
    maxModelCalls: 3
```

---

36. ChangeSet

Fundamental source mutation primitive.

```
type ChangeSet struct {
    ID             UUID
    HumanID        string
    InitiativeID   UUID
    TaskID         UUID
    BaseSnapshotID string
    Status         ChangeSetStatus
    PatchDigest    Digest
    ReviewID       *UUID
    GitCommitSHA   *string
    CreatedByRunID UUID
}
```

---

37. Worker Isolation

Git:

```
worktree
```

Non-Git:

```
filesystem snapshot/copy
```

Workers never receive the canonical workspace as a writable execution root.

---

38. ChangeSet State

```
draft
 ↓
checks_running
 ├──────────────┐
 ▼              ▼
review_required rejected
 │
 ├→ changes_requested
 │      ↓
 │   superseded
 │
 └→ approved
       ↓
    applying
    ├──────┐
    ▼      ▼
 applied conflicted
```

Formal transition matrix is required in source code and docs.

---

39. ApplyJournal

Filesystem + SQLite cannot be committed atomically.

Therefore canonical ChangeSet application requires a durable journal.

```
prepared
workspace_mutated
committed
rolled_back
recovery_required
```

Apply sequence:

1. lock canonical workspace
2. verify base
3. create rollback snapshot
4. persist `prepared`
5. apply patch
6. verify hashes
7. persist `workspace_mutated`
8. transactionally mark ChangeSet applied + event + journal committed
9. release lock
10. garbage-collect rollback later

Startup reconciliation handles incomplete journals.

---

40. Revert

Before acceptance:

```
exact rollback
```

After later changes:

```
new inverse ChangeSet
```

Never restore an old snapshot over newer user work.

---

41. Git

Git is a capability.

States:

```
none
local
remote
```

Automatic detection occurs:

- project open
- executor start
- ChangeSet apply
- structural filesystem changes

No automatic:

- git init
- remote creation
- push

---

42. Context Compiler

v0.1 is deterministic-first.

Priority:

```
immutable policy
project policy
role contract
stage instructions
task
direct requirements
direct decisions
required artifacts
explicitly related files
standards
history
```

Avoid embeddings initially.

---

43. Context Trust

Trust classes:

```
system
policy
human_approved
reviewed_agent_output
```

```
unreviewed_agent_output
external_untrusted
```

Agent-generated content does not automatically become trusted merely because it originated inside idea-harness.

---

44. Context Provenance

Each entry stores:

- source
- digest
- trust
- priority
- inclusion reason
- selector
- transformation
- original/final size

`idea inspect context RUN-123` exposes all of this.

---

45. Structured Output Pipeline

Centralized:

```
model
 ↓
provider-native schema enforcement
 ↓
validation
 ↓
bounded repair
 ↓
validated object
```

Raw and repaired content are both retained by digest.

---

46. Model Provider Interface

Capabilities include:

```
structured_output
tools
streaming
reasoning controls
context window
max output
```

Role/Stage requirements are checked against provider capabilities during config validation.

Provider-specific options remain opaque extension fields.

---

47. Model Fallback

Must be explicit.

RunManifest records:

```
requested model
actual model
fallback reason
```

No silent fallback.

---

48. Executor

Initial external executor abstraction should be validated through OMP before generalization.

```go
type Executor interface {
    Name() string
    Capabilities(context.Context) ExecutorCapabilities
    Start(context.Context, ExecutionRequest) (ExecutionAttempt, error)
    Status(context.Context, UUID) (ExecutionStatus, error)
    Cancel(context.Context, UUID) error
```

```
    Collect(context.Context, UUID) (ExecutionResult, error)
}
```

---

49. Executor Trust

```
internal
local_external
remote_external
```

OMP begins as:

```
local_external
```

Executor trust may reduce context, repository, network, or secret access.

---

50. Progress

Executors emit:

```
heartbeat
phase
tool activity
current action
```

Do not expose chain-of-thought.

Runtime should show:

```
Planning
Editing
Testing
Waiting
Reviewing
Finished
```

---

51. Security Boundaries

Sandbox

OS-level blast-radius control.

Capability Broker

Semantic governance for mediated actions.

Do not confuse them.

Repository-controlled scripts are arbitrary code once executed.

Therefore:

```
policy controls intent
sandbox controls containment
```

---

52. Sandbox Modes

```
container
local_restricted
host_approved
```

Default:

```
container
```

macOS/iOS-specific tasks may require host-approved execution.

High-risk host access requires explicit authorization.

---

53. Network v0.1

Sandbox:

```
disabled
enabled
```

Prefer controlled web/research Tools rather than arbitrary shell egress.

Fine-grained egress control is P1+.

---

54. Filesystem Broker

All requests include Project ID.

Defend against:

- traversal
- symlink escape
- cross-project access
- forbidden paths
- oversized files

---

55. Secret Broker

Secrets are never persisted as ordinary content.

Resolve from:

- environment
- OS keychain
- future secret manager adapters

Secret access is Project-scoped.

---

56. Output Redaction

Before persistence inspect:

- logs
- model responses
- tool responses
- review text
- event textual payloads

for:

- known secret values
- common credential formats
- configured sensitive markers

Redact before writing, not merely on display.

---

57. Skills

Skills are non-executable packages.

v0.1:

```
local directory
Git source pinned to commit
```

Lockfile records:

```
source
revision
digest
```

Integrity mismatch blocks use.

---

58. Tools

Executable extensions are separate from Skills.

Do not build a broad plugin ecosystem before actual integrations validate the abstraction.

---

59. Policy Packs

Declarative only in v0.1.

No arbitrary policy scripting language.

---

60. Local API

Product-oriented rather than database-CRUD-oriented.

Examples:

```
POST /initiatives
GET  /initiatives/:id
GET  /initiatives/:id/home
GET  /initiatives/:id/changed

POST /initiatives/:id/start
POST /initiatives/:id/pause

GET  /decisions
GET  /reviews/:id
POST /reviews/:id/findings/:id/disposition

GET  /trace/:type/:id
GET  /impact/:type/:id
GET  /why/:type/:id
GET  /explain/:type/:id

GET  /operations/:id
```

v0.x API should be explicitly unstable/internal.

---

61. API Security

Daemon binds localhost.

CLI/UI authenticate with local session credentials.

Remote binding opt-in only.

Human principals and agent principals are distinct.

Agents cannot invoke human-only approval endpoints.

---

62. CLI

Primary:

```
idea init
idea dev
idea new
idea status
idea changed

idea decisions
idea reviews
idea approvals

idea explain
idea why
idea impact
idea trace

idea inspect run
idea inspect context

idea git status
idea git init
idea git connect

idea skill list
idea skill add

idea config validate
idea doctor
```

Structured state commands support:

```
--json
```

---

63. UI Architecture

Recommended:

```
React
TypeScript
Vite
TanStack Query
React Router
SSE
generated API client
```

Server remains authoritative.

Avoid duplicating orchestration state machines in the client.

---

64. Primary UI Navigation

```
Ideas
Decisions
Activity

Runtime
Settings
```

Reviews generally appear contextually in an Initiative or as Decision/Attention items.

Do not expose a dozen infrastructure-oriented top-level sections.

---

65. Initiative Home API

Return a purpose-built projection:

```go
type InitiativeHome struct {
    CurrentUnderstanding CurrentUnderstanding
    RecommendedNextMove  *RecommendedAction
    Decisions            []DecisionSummary
    OpenQuestions        []OpenQuestion
    Changed              []SemanticChange
    MacroPhase           MacroPhase
    DesignSubstates      []StageSummary
    Risk                 RiskSummary
}
```

Do not force the UI to reconstruct this from dozens of generic resources.

---

66. Decisions

Decision UI data should synthesize:

- question
- alternatives
- expert perspectives
- recommendation
- rationale
- impact
- authority needed

The Orchestrator Agent may synthesize expert perspectives but cannot hide disagreements.

---

67. Activity Timeline

Timeline should ship before sophisticated graph visualization.

Events should group into human-readable activity such as:

```
Research completed
PRD revised
Architecture challenged requirement
ChangeSet rejected
Requirement verified
```

Raw events available as expert drill-down.

---

68. Runtime

Runtime is a power-user/operator view.

Show:

- active Run
- role
- model
- executor

- phase
- elapsed time
- heartbeat
- task
- cost
- current operational action

No internal chain-of-thought.

---

69. Explain / Why / Impact / Trace

Explain

Describe the object.

Why

Show causal origin.

Impact

Traverse downstream dependencies.

Trace

Show provenance path.

LLM synthesis may improve wording.

Underlying causal/graph skeleton must be deterministic.

---

70. Requirements Verification

A Requirement cannot be considered verified merely because implementation tests exist.

Verification Plan determines evidence requirements.

Each mandatory AcceptanceCriterion requires valid VerificationEvidence.

Initiative completion gate checks all required coverage.

---

71. Autonomy

Autonomy presets compile into explicit policies.

Example:

```
Supervised
  human approves execution plan
  human approves high-risk ChangeSets

Standard
  human only for critical risk or escalation

High
  human only for policy-required escalation
```

The UI must explain what the selected level changes.

---

72. Error Taxonomy

Infrastructure:

```
provider_rate_limit
provider_timeout
provider_unavailable
tool_timeout
executor_crash
worker_lost
```

Semantic:

```
test_failure
review_failure
requirements_conflict
dependency_invalidated
workspace_conflict
```

Governance:

```
permission_denied
budget_exceeded
review_non_convergence
human_approval_required
```

State:

```
invalid_configuration
state_corruption
skill_integrity_failure
```

Recovery behavior differs by class.

---

73. RunManifest

Every Run records:

- project
- workspace snapshot
- config hash
- workflow hash
- role version
- requested/actual model
- context digest
- skills
- tools
- executor version
- start/end time
- result digest

Immutable after completion.

---

74. Metrics

From Milestone 1 instrument:

```
idea→reviewed PRD
idea→first code
decision latency
ChangeSet review latency
```

```
requirement coverage
verification coverage
review precision
defect escapes
human intervention
human overrides
provider failures
retry rate
cost/initiative
context efficiency
```

---

75. Testing

Unit:

- transitions
- graph traversal
- policy
- risk
- review logic
- context priority

Component:

- SQLite
- BlobStore
- Git
- sandbox
- brokers
- OMP

Integration:

- full stage lifecycle
- review revision
- ChangeSet lifecycle
- config reload
- Git appearing mid-project
- external workspace changes

E2E:

```
empty folder
→ idea init
```

```
→ idea new
→ reviewed PRD
→ reviewed design/plan
→ OMP
→ reviewed ChangeSet
→ canonical apply
→ verification
```

---

76. Threat Model

Before secure execution ships, create a dedicated threat model covering:

- malicious model output
- prompt injection
- malicious repository
- malicious Skill
- malicious Tool
- malicious Executor
- local API attacks
- credential exposure
- sandbox escape
- cross-project leakage
- state tampering
- audit tampering
- malicious agent-to-agent messages
- supply-chain compromise

---

77. Delivery

Milestone 1

Idea → reviewed PRD

Must include thin UI.

Milestone 2

PRD → reviewed execution plan

Adds Design, Quality, Security, Product Graph.

Milestone 3

Plan → safe reviewed ChangeSet

Adds OMP, sandbox, ChangeSet, ApplyJournal, Git integration.

Milestone 4

Requirement → verified product

Adds verification coverage, impact, staleness, Explain/Why/Trace.

Build vertically.

---

78. Final Architecture Principle

```
Agents reason.

The engine governs.

Experts challenge.

The sandbox contains.

ChangeSets isolate.

The graph remembers.

Evidence verifies.

The UI synthesizes.

The human decides what matters.
```